WTC_ AI Curriculum

# Exercise: README and User Guide Documentation

## Overview

In this exercise, you'll practice using AI to create comprehensive project documentation, including README files and user guides. You'll apply different prompt strategies to document a project's features, installation process, and usage instructions.

## Exercise Options

You have several options for this exercise:

1. **Document a JavaScript/Node.js project**: Use one of the provided project examples
2. **Document a Python project**: Use one of the provided project examples
3. **Document the Task Manager project**: Use the Task Manager system from the Code Comprehension exercise.

## Instructions

1. Select the project you want to document
2. Apply Prompt 1 to generate a comprehensive README file
3. Review the README for completeness and accuracy
4. Apply Prompt 2 to create a step-by-step guide for one of the project's features
5. Apply Prompt 3 to create an FAQ document for the project
6. Compare your documentation with a partner and provide feedback

## Prompt Template Reminders

### Prompt 1: Project README Generation

```
Please create a comprehensive README.md file for my project based on the
following information:

Project name: [Name]
Description: [Brief description]
Key features:
- [Feature 1]
```

```
- [Feature 2]
- [etc.]

Technologies used: [List technologies, frameworks, languages]
Installation requirements: [List prerequisites]

The README should include:
1. Clear project title and description
2. Installation instructions
3. Basic usage examples
4. Features overview
5. Configuration options
6. Troubleshooting section
7. Contributing guidelines
8. License information

Code structure overview:
[Briefly describe main directories/files or paste project structure]
```

### Prompt 2: Step-by-Step Guide Creation

```
Please create a step-by-step guide for how to [specific task/process] in our
application.

The guide should:
1. Start with any prerequisites or required access
2. Break down the process into clear, numbered steps
3. Include screenshots or code blocks where indicated [Placeholder]
4. Highlight potential issues or common mistakes
5. End with a troubleshooting section for common problems

Process overview:
[Describe the task or provide notes on what the user needs to accomplish]

User experience level: [Beginner/Intermediate/Advanced]
```

### Prompt 3: FAQ Document Generation

```
Please help me create a comprehensive FAQ document for
[project/feature/product].

Basic information:
- [Brief description of what we're creating FAQs for]
- [Target audience]
- [Any specific areas to focus on]

Please include:
1. Questions about getting started
2. Questions about common features and functionality
```

```
3. Questions about troubleshooting common issues
4. Questions about [specific area of interest]

For each question, provide a clear, concise answer that would be helpful to our
users.

Known issues or common user questions:
[List any specific questions or issues to include]
```

## JavaScript/Node.js Project Examples

### Example 1: Event Management System

```
Project name: EventHub
Description: A web application for creating and managing events, with attendee
registration and ticket sales

Key features:
- Event creation and management
- Attendee registration and ticket sales
- Email notifications and reminders
- Event check-in system
- Analytics dashboard

Technologies used: Node.js, Express, MongoDB, React, Redux, Stripe API
Installation requirements: Node.js 14+, MongoDB, npm or yarn

Project structure:
/client - React frontend application
/server - Express backend API
/server/routes - API route definitions
/server/controllers - Business logic
/server/models - MongoDB schemas
/server/middleware - Auth and validation middleware
/client/src/components - React components
/client/src/pages - Page components
/client/src/redux - Redux store, actions, and reducers
/client/src/utils - Utility functions
```

### Example 2: Content Management System

```
Project name: SimpleCMS
Description: A lightweight content management system for blogs and small
websites

Key features:
- User-friendly content editor with markdown support
- Media library for images and documents
```

```
- User management with different permission levels
- Custom themes and templates
- SEO optimization tools
- Analytics integration

Technologies used: Node.js, Express, MySQL, Handlebars, jQuery
Installation requirements: Node.js 12+, MySQL 5.7+

Project structure:
/admin - Admin dashboard interface
/controllers - Request handlers
/models - Database models
/routes - Route definitions
/views - Handlebars templates
/public - Static assets
/uploads - User uploaded content
/config - Configuration files
/utils - Utility functions
```

### Example 3: Task Management CLI Tool

```
Project name: TaskCLI
Description: A command-line task management tool for developers

Key features:
- Create, update, and track tasks from the terminal
- Organize tasks into projects and tags
- Set priorities and deadlines
- Daily summary emails
- GitHub integration
- Time tracking

Technologies used: Node.js, Commander.js, Inquirer.js, SQLite
Installation requirements: Node.js 10+, npm

Project structure:
/bin - CLI entry point
/commands - Command implementations
/models - Data models
/utils - Utility functions
/db - Database handlers
/services - External service integrations
/config - Configuration files
```

## Python Project Examples

### Example 1: Data Analysis Framework

```
Project name: DataInsight
Description: A data analysis and visualization framework for scientific data

Key features:
- Data import from various sources (CSV, Excel, SQL, APIs)
- Data cleaning and transformation tools
- Statistical analysis functions
- Interactive visualizations
- Report generation
- Export capabilities

Technologies used: Python, Pandas, NumPy, Matplotlib, Plotly, Jupyter
Installation requirements: Python 3.8+, pip

Project structure:
/datainsight - Main package
/datainsight/io - Data import/export modules
/datainsight/transform - Data transformation tools
/datainsight/analysis - Analysis algorithms
/datainsight/viz - Visualization tools
/datainsight/report - Report generation
/examples - Example notebooks
/tests - Unit tests
```

### Example 2: Web Scraping Framework

```
Project name: ScrapeMaster
Description: A flexible web scraping framework with scheduling and data
processing

Key features:
- Define scrapers using a simple configuration format
- Scheduled scraping jobs
- Proxy rotation and user agent switching
- CAPTCHA handling
- Data extraction using CSS selectors or XPath
- Export to various formats (CSV, JSON, Database)
- Webhook notifications

Technologies used: Python, Scrapy, BeautifulSoup, SQLAlchemy, Celery
Installation requirements: Python 3.7+, Redis, pip

Project structure:
/scrapemaster - Main package
/scrapemaster/scrapers - Scraper definitions
/scrapemaster/extractors - Data extraction tools
/scrapemaster/processors - Data processing pipelines
/scrapemaster/exporters - Data export modules
/scrapemaster/scheduler - Job scheduling
/scrapemaster/utils - Utility functions
```

```
/config - Configuration files
/examples - Example scrapers
```

### Example 3: Machine Learning Toolkit

```
Project name: MLToolkit
Description: A toolkit for building, training, and deploying machine learning
models

Key features:
- Simplified interfaces for common ML algorithms
- Automated data preprocessing
- Hyperparameter optimization
- Model evaluation and comparison tools
- Model serialization and deployment helpers
- Visualization of model performance
- Explainable AI components

Technologies used: Python, Scikit-learn, TensorFlow, Pandas, Matplotlib
Installation requirements: Python 3.8+, pip

Project structure:
/mltoolkit - Main package
/mltoolkit/preprocessing - Data preprocessing tools
/mltoolkit/models - Model implementations
/mltoolkit/training - Training utilities
/mltoolkit/evaluation - Evaluation metrics and tools
/mltoolkit/visualization - Visualization utilities
/mltoolkit/deployment - Deployment helpers
/examples - Example applications
/notebooks - Jupyter notebooks
/tests - Unit tests
```

## Example Documentation Outputs

To give you an idea of what your documentation might look like, here are examples for different types of documentation:

### README Example

```
# TaskCLI

A powerful command-line task management tool designed for developers who prefer
to stay in the terminal.

![TaskCLI Demo](https://placeholder.com/taskCLI-demo.gif)

## Features
```

```
- **Task Management**: Create, update, and track tasks directly from your
terminal
- **Organization**: Group tasks into projects and add tags for easy filtering
- **Prioritization**: Set priorities and deadlines to stay focused on what
matters
- **Notifications**: Receive daily summary emails of pending tasks
- **GitHub Integration**: Link tasks to GitHub issues and pull requests
- **Time Tracking**: Track time spent on different tasks and generate reports

## Installation

### Prerequisites

- Node.js 10.0 or higher
- npm or yarn

### Quick Install

```bash
# Install globally
npm install -g task-cli

# Verify installation
taskcli --version
```

### Manual Installation

1. Clone the repository:
```bash
git clone https://github.com/yourusername/task-cli.git
cd task-cli
```

2. Install dependencies:
```bash
npm install
```

3. Link the package globally:
```bash
npm link
```

## Configuration

On first run, TaskCLI will create a configuration file at
`~/.taskcli/config.json`. You can edit this file directly or use the
configuration command:
```

```bash
taskcli config set email your.email@example.com
taskcli config set githubToken your-github-token
```

### Available Configuration Options

| Option | Description | Default |
|---|---|---|
| `email` | Email for notifications | None |
| `githubToken` | GitHub Personal Access Token | None |
| `defaultProject` | Default project for new tasks | "inbox" |
| `reminderTime` | Time for daily reminder emails | "08:00" |
| `dateFormat` | Date format for display | "YYYY-MM-DD" |

## Usage

### Basic Commands

```bash
# Create a new task
taskcli add "Implement new feature"

# List all tasks
taskcli list

# Mark a task as complete
taskcli complete 42

# View task details
taskcli show 42
```

### Managing Projects

```bash
# Create a new project
taskcli project add "Website Redesign"

# List all projects
taskcli project list

# Add a task to a project
taskcli add "Design homepage" --project "Website Redesign"
```

### Using Tags

```bash
# Add tags to a task
taskcli tag 42 --add urgent frontend
```

```
# List tasks with specific tags
taskcli list --tags urgent
```

### Time Tracking

```bash
# Start tracking time for a task
taskcli timer start 42

# Stop the current timer
taskcli timer stop

# View time report
taskcli report time --from 2023-01-01 --to 2023-01-31
```

## Troubleshooting

### Common Issues

#### Command Not Found

If you see `command not found` after installation, make sure your npm global bin directory is in your PATH.

#### Authentication Errors

For GitHub integration issues, verify your token has the correct permissions (repo scope).

#### Database Errors

If you see database errors, try resetting the database:

```bash
taskcli db:reset
```

## Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add some amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

## License

```
This project is licensed under the MIT License - see the [LICENSE](LICENSE) file
for details.

## Acknowledgments

- [Commander.js](https://github.com/tj/commander.js/) for CLI argument parsing
- [Inquirer.js](https://github.com/SBoudrias/Inquirer.js/) for interactive
prompts
```

### Step-by-Step Guide Example

```
# How to Set Up GitHub Integration in TaskCLI

This guide will walk you through the process of setting up GitHub integration
for TaskCLI, allowing you to link tasks to GitHub issues and pull requests.

## Prerequisites

Before you begin, make sure you have:

- TaskCLI installed (version 2.0 or higher)
- A GitHub account
- Repository access for the repositories you want to integrate with

## Step 1: Create a GitHub Personal Access Token

1. Log in to your GitHub account
2. Go to Settings > Developer settings > Personal access tokens
3. Click "Generate new token"
4. Give your token a name, e.g., "TaskCLI Integration"
5. Select the following scopes:
- `repo` (Full control of private repositories)
- `read:user` (Read user information)
6. Click "Generate token"
7. **IMPORTANT**: Copy the generated token immediately. GitHub will only show it
once!

[PLACEHOLDER FOR SCREENSHOT: GitHub token generation screen]

## Step 2: Configure TaskCLI with Your Token

1. Open your terminal
2. Run the following command, replacing `your-token` with the token you just
generated:
```bash
taskcli config set githubToken your-token
```
3. Verify the token is set correctly:
```bash
```
```

```
taskcli config list
```
You should see your token (partially masked) in the output.

## Step 3: Link a GitHub Repository

1. Create a new project or use an existing one:
```bash
taskcli project add "Website Frontend"
```

2. Link the project to a GitHub repository:
```bash
taskcli project link "Website Frontend" --repo "username/repository"
```

3. Verify the link is working:
```bash
taskcli project show "Website Frontend"
```
You should see the repository information in the output.

## Step 4: Link Tasks to GitHub Issues

### Creating a Task from an Issue

1. Import an existing GitHub issue as a task:
```bash
taskcli github import --repo "username/repository" --issue 42
```

2. Or reference an issue when creating a new task:
```bash
taskcli add "Fix navigation bug" --github-issue 42
```

### Linking an Existing Task to an Issue

1. Find the ID of your task:
```bash
taskcli list
```

2. Link the task to a GitHub issue:
```bash
taskcli link 24 --github-issue 42
```

## Step 5: Using GitHub Integration Features

1. Pull updates from GitHub issues:

```bash
taskcli github sync
```
This will update the status of your tasks based on the linked issues.

2. Push task updates to GitHub:
```bash
taskcli complete 24 --push-github
```
This will mark the linked GitHub issue as closed.

3. View GitHub information for a task:
```bash
taskcli show 24 --github-details
```

## Troubleshooting

### Token Authentication Failed

If you see "Authentication failed" errors:

1. Verify your token has not expired
2. Check that it has the correct permissions
3. Try regenerating a new token and updating your configuration

### Repository Not Found

If you get "Repository not found" errors:

1. Check the repository name is correct (username/repository)
2. Verify you have access to the repository
3. Ensure your token has permissions for that repository

### Changes Not Syncing

If changes aren't syncing between TaskCLI and GitHub:

1. Run `taskcli github sync --force` to force a full synchronization
2. Check if there are any network connectivity issues
3. Look for any error messages in the TaskCLI log (`taskcli log show`)

## Next Steps

Now that you've set up GitHub integration, you might want to explore:

- Setting up automated daily syncs with `taskcli cron setup`
- Creating custom GitHub issue templates for tasks
- Using webhooks for real-time updates (advanced)

```
If you have any questions or run into issues, please refer to the full
documentation or open an issue on our GitHub repository.
```

## FAQ Document Example

MARKDOWN |

```
# TaskCLI Frequently Asked Questions

## Getting Started

### What is TaskCLI?
TaskCLI is a command-line task management tool designed for developers who
prefer to work in the terminal. It allows you to create, manage, and track tasks
without leaving your development environment.

### How do I install TaskCLI?
You can install TaskCLI globally using npm:
```bash
npm install -g task-cli
```
After installation, you can run commands using the `taskcli` command.

### What are the system requirements?
TaskCLI requires Node.js 10.0 or higher and npm. It works on Windows, macOS, and
Linux.

### How do I get started after installation?
After installing, run `taskcli init` to set up your configuration. Then create
your first task with `taskcli add "My first task"`.

## Basic Usage

### How do I create a new task?
Use the `add` command followed by the task description in quotes:
```bash
taskcli add "Implement login page"
```

### How do I view my tasks?
Use the `list` command to see all active tasks:
```bash
taskcli list
```
You can filter by project, tag, or status.

### How do I mark a task as complete?
Use the `complete` command followed by the task ID:
```bash
taskcli complete 42
```
```

````
### Can I set due dates for tasks?
Yes, you can add a due date when creating a task:
```bash
taskcli add "Write documentation" --due 2023-12-31
```
Or update an existing task:
```bash
taskcli update 42 --due 2023-12-31
```

## Projects and Tags

### What are projects in TaskCLI?
Projects help you organize related tasks. Think of them as folders for your
tasks.

### How do I create a new project?
Use the `project add` command:
```bash
taskcli project add "Website Redesign"
```

### How do I add a task to a project?
You can specify the project when creating a task:
```bash
taskcli add "Design homepage" --project "Website Redesign"
```
Or move an existing task:
```bash
taskcli update 42 --project "Website Redesign"
```

### What are tags and how do I use them?
Tags are labels that help you categorize tasks across projects. Add tags when
creating a task:
```bash
taskcli add "Fix bug" --tags urgent backend
```
Or add tags to an existing task:
```bash
taskcli tag 42 --add urgent frontend
```

## Time Tracking

### How do I track time spent on a task?
Start tracking time with:
```bash
taskcli timer start 42
```
And stop the timer with:
````

```bash
taskcli timer stop
```

### Can I manually add time entries?
Yes, use the `timer add` command:
```bash
taskcli timer add 42 --minutes 30 --date 2023-01-15
```

### How do I view time reports?
Generate reports with the `report` command:
```bash
taskcli report time --from 2023-01-01 --to 2023-01-31
```
You can filter by project, tag, or task.

## GitHub Integration

### How do I link TaskCLI to GitHub?
Set up your GitHub token with:
```bash
taskcli config set githubToken your-github-token
```

### Can I create a task from a GitHub issue?
Yes, import an issue with:
```bash
taskcli github import --repo "username/repository" --issue 42
```

### Do task status changes sync with GitHub?
Yes, when you complete a task linked to a GitHub issue, you can update the issue simultaneously:
```bash
taskcli complete 42 --push-github
```

## Troubleshooting

### TaskCLI command not found after installation
Make sure your npm global bin directory is in your PATH environment variable. You can find it by running `npm bin -g`.

### Database errors or corruption
Reset your database with:
```bash
taskcli db:reset
```
This will preserve your tasks but rebuild the database structure.

````
### Email notifications not working
Verify your email configuration with:
```bash
taskcli config list
```
Make sure your email settings are correct and try testing with:
```bash
taskcli email:test
```

### How do I completely uninstall TaskCLI and all its data?
Uninstall the package with:
```bash
npm uninstall -g task-cli
```
Then remove the data directory at `~/.taskcli/` or `%APPDATA%\taskcli\` on
Windows.

## Advanced Features

### Can I create recurring tasks?
Yes, use the `--recurring` flag when creating a task:
```bash
taskcli add "Weekly report" --recurring weekly --weekday friday
```

### How do I back up my TaskCLI data?
Export your data with:
```bash
taskcli export --format json --output backup.json
```
You can import it later with:
```bash
taskcli import backup.json
```

### Can I use TaskCLI on multiple computers?
Yes, you can sync your data using the cloud sync feature:
```bash
taskcli sync enable
```
This requires setting up a sync provider in your configuration.

### Is there an API for integrating with other tools?
Yes, you can start the API server with:
```bash
taskcli api start
```
The API documentation is available at http://localhost:3000/api-docs when the
server is running.
````

## What to Submit

At the end of the exercise, you should have:

1. The project information you chose to document
2. The comprehensive README you generated using Prompt 1
3. The step-by-step guide you created using Prompt 2
4. The FAQ document you created using Prompt 3

Be prepared to share what you learned about:

- Which aspects of the project were most challenging to document
- How you adjusted your prompts to get better results
- What you learned about document structure and organization
- How you would incorporate this approach into your development workflow